

Two bugs in GuidedMERS, fixed and measured

PhyloPOMP.jl, atpabuser-devel branch, 2026-09-22

Table of contents

Run this now	1
Bug 1 — within-host fork boost missing a factor of 2	2
Live check 1: the compiler’s independent formula agrees	3
Live check 2: the end-to-end likelihood gap	3
Bug 2 — regular within-host births charge no “no visible fork” factor	5
The full 274-tip tree: measured, not live	6
Appendix: reproducing the full-tree comparison	6
Summary	7

Run this now

```
quarto render docs/guided_mers_bugfix/mers_guided_bugfix.qmd
```

Self-contained project (docs/guided_mers_bugfix/Project.toml), points at this checkout of PhyloPOMP (../..) and lets PartiallyObservedMarkovProcesses (POMP) resolve from the registry — the same 0.10.0 that every number below was measured against. PhyloPOMP re-exports POMP (pfilter, logLik, logmeanexp, mif, ...), so nothing below needs a POMP. prefix; Pkg.status() is the one place the version is checked directly.

Upstream (kingaa/PhyloPOMP.jl, commit 25039d0) rewrote src/examples/mers_guided.jl (module GuidedMERS, the two-host Camel/Human MERS-CoV filter). The rewrite has two likelihood bugs. Both are fixed in this branch’s copy of the file. This document shows each bug, shows the fix is the derivable correct formula (not just a plausible-looking patch), and measures what the fix is worth on the real 274-tip MERS tree.

```

import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()

using PhyloPOMP
using PhyloPOMP.GuidedMERS
using PhyloPOMP: full_transitions, ForkTransition, IdentityTransition,
                InlineSameDemeTransition
using Random: seed!

Pkg.status()

```

```

Activating project at `~/Desktop/Projects/PhyloPOMP.jl/docs/guided_mers_bugfix`
Status `~/Desktop/Projects/PhyloPOMP.jl/docs/guided_mers_bugfix/Project.toml`
 [31c24e10] Distributions v0.25.131
 [32433216] PartiallyObservedMarkovProcesses v0.10.0
 [d3093253] PhyloPOMP v0.0.13-3 `../..`
 [9a3f8284] Random v1.11.0

```

That `Pkg.status()` cell is the receipt: everything below ran against exactly the package versions it names, not a description of an earlier run.

Bug 1 — within-host fork boost missing a factor of 2

`singular_branch!` (`src/examples/mers_guided.jl:184`, the branch-point proposal fired when an observed coalescent node forks two lineages within the same host) charged, on a camel→camel fork:

```
ll -= log(I_c*(I_c-1))      # upstream, wrong
```

The correct charge is $-\log(\text{binomial}(I_c, 2))$, i.e.

```
ll -= log(I_c*(I_c-1)/2)    # this branch, src/examples/mers_guided.jl:187
```

Why. A camel→camel fork means two of the I_c indistinguishable camel infecteds turn out to be the two observed child lineages. The number of *ordered* pairs is $I_c*(I_c-1)$; the event “these two particular individuals” is a draw of an *unordered* pair, $\text{binomial}(I_c, 2) = I_c*(I_c-1)/2$ of them. The human→human fork (line 219) has the identical bug and fix.

Live check 1: the compiler's independent formula agrees

src/examples/mgp_*.jl computes transition probabilities for every MGP event from first principles (KLI reduction, full_transitions), with no code shared with mers_guided.jl. Its ForkTransition.phi for a transmission_cc fork, in exact rational arithmetic:

```
tcc = PhyloPOMP.MERS.events[findfirst(e -> e.name == :transmission_cc, PhyloPOMP.MERS.e

for Ic in (2, 3, 5, 8)
    ts = full_transitions(tcc, [2, 0], [Ic, 0])
    fk = only(filter(t -> t isa ForkTransition, ts))
    expected = 2 // (Ic * (Ic - 1))
    println("I_c=$Ic    IR phi = $(fk.phi)    2/(I_c(I_c-1)) = $expected    match = $(fk.p
end
```

```
I_c=2    IR phi = 1//1    2/(I_c(I_c-1)) = 1//1    match = true
I_c=3    IR phi = 1//3    2/(I_c(I_c-1)) = 1//3    match = true
I_c=5    IR phi = 1//10   2/(I_c(I_c-1)) = 1//10   match = true
I_c=8    IR phi = 1//28   2/(I_c(I_c-1)) = 1//28   match = true
```

$2/(I_c(I_c-1))$ is exactly $1/\text{binomial}(I_c, 2)$. Every row matches. mers_naive.jl (the independent hand-coded oracle, untouched by this branch's changes) charges the same /2 on its own camel-camel and human-human fork lines.

Live check 2: the end-to-end likelihood gap

An all-camel, 4-tip tree with $\beta_{hc} = \beta_{ch} = 0$ forces every one of its 3 internal nodes to be a camel→camel fork — the bug fires on every internal node, nowhere else. Expected gap between the buggy and fixed kernel: $3 * \log(2)$.

```
nwk = "((( [&&PhyloPOMP deme=camel]1:1.0, [&&PhyloPOMP deme=camel]2:1.0):0.5, " *
      "[&&PhyloPOMP deme=camel]3:1.2):0.3, [&&PhyloPOMP deme=camel]4:1.9);"
g = parse_newick(nwk, demes = GuidedMERS.Demes)
m = fsmarkov(GuidedMERS.Camel => 0.5, GuidedMERS.Human => 0.5,
             (GuidedMERS.Camel, GuidedMERS.Human) => 0.01)
common = (β_cc = 3.0, β_ch = 0.0, β_hc = 0.0, β_hh = 3.0, γ_c = 1.0, γ_h = 1.0,
          χ_c = 1.0, χ_h = 1.0, B_c = 0.0, B_h = 0.0,
          S_c0 = 1.0, S_h0 = 1.0, I_c0 = 0.5, I_h0 = 0.0, N_c = 60, N_h = 60)

p_fixed = GuidedMERS.filter_pomp(g, m; common...)
seed!(1)
ll_fixed = [logLik(pfilter(p_fixed, Np = 2000)) for _ in 1:15]
est_fixed, se_fixed = logmeanexp(ll_fixed, se = true)
```

```

## Reinstall the upstream (buggy) formula, in place, on the SAME module –
## filter_pomp's rprocess closure calls singular_branch! by name, so
## redefining it here changes every subsequent GuidedMERS.filter_pomp call
## without rebuilding anything else. Nothing later in this document reuses
## GuidedMERS.filter_pomp, so no restore step is needed.
@eval GuidedMERS begin
  singular_branch!(
    gennode, guidenode, cols, (;S_c, I_c, S_h, I_h);
    β_cc, β_hh, β_hc, β_ch, N_c, N_h, _...,
  ) = begin
    if guidenode.parlin ∈ cols[Camel]
      live = true
      k, _, p = rcateg(
        [β_cc*S_c*I_c*guidenode.present[1,1]*guidenode.present[1,2],
         β_hc*S_h*I_c*guidenode.present[1,1]*guidenode.present[2,2],
         β_hc*S_h*I_c*guidenode.present[2,1]*guidenode.present[1,2]], true)
      if k==1
        ll = log(β_cc*S_c*I_c/N_c) - log(p)
        fork!(cols, Camel, guidenode.parlin, (Camel, Camel), guidenode.chillins)
        S_c -= 1; I_c += 1
        ll -= log(I_c*(I_c-1))          # upstream formula: missing /2
      else
        live = false; ll = Prob(-Inf)
      end
    else
      live = false; ll = Prob(-Inf)
    end
    ;live, ll, (;S_c, I_c, S_h, I_h)
  end
end

p_buggy = GuidedMERS.filter_pomp(g, m; common...)
seed!(1)
ll_buggy = [logLik(pfilter(p_buggy, Np = 2000)) for _ in 1:15]
est_buggy, se_buggy = logmeanexp(ll_buggy, se = true)

gap = est_fixed - est_buggy
gap_se = sqrt(se_fixed^2 + se_buggy^2)
println("fixed    logLik = $(round(est_fixed, digits=3)) ± $(round(se_fixed, digits=3))")
println("buggy    logLik = $(round(est_buggy, digits=3)) ± $(round(se_buggy, digits=3))")
println("gap      = $(round(gap, digits=3)) ± $(round(gap_se, digits=3))    (expect

```

fixed logLik = -26.324 ± 0.49
buggy logLik = -28.404 ± 0.49
gap = 2.079 ± 0.692 (expect 3*log(2) = 2.079)

Bug 2 — regular within-host births charge no “no visible fork” factor

`regular_transmission_cc!/regular_transmission_hh!` — the proposal used when a camel-camel (or human-human) birth happens in an interval with *no* observed genealogy event — returned a flat zero log-weight upstream. A within-deme birth that occurs silently (no branch point in the data) must be down-weighted by the chance it did **not** join two of the ℓ currently tracked lineages:

```
log(1 - ellC*(ellC-1)/I_c/(I_c-1))    # this branch, src/examples/mers_guided.jl:267-27
```

versus upstream’s `zero(Prob)`. This only bites when $\ell \geq 2$ — the 3-tip tree above never has two tracked camel lineages alive at once, so it under-tests this bug. The underlying claim is the Chu-Vandermonde identity (already checked, independently, in `test/kli_mers_compiled_test.jl`’s “TCC/THH weighted aggregate” testset): the no-visible-fork mass equals

$$\Phi_{\text{identity}} + \ell \cdot \Phi_{\text{inline}} = 1 - \binom{\ell}{2} \Phi_{\text{fork}}$$

for the same `transmission_cc` event the IR already computed above, at a state with two tracked camel lineages ($\ell=2$):

```
ts = full_transitions(tcc, [2, 0], [5, 0])    # ell_C = 2, I_c = 5

phi_of(T) = (i = findfirst(t -> t isa T, ts); isnothing(i) ? 0//1 : ts[i].phi)
phi_id = phi_of(IdentityTransition)
phi_inl = phi_of(InlineSameDemeTransition)
phi_fork = phi_of(ForkTransition)
ell = 2

lhs = phi_id + ell * phi_inl
rhs = 1 - binomial(ell, 2) * phi_fork
println("Phi_identity + ell*Phi_inline = $lhs")
println("1 - binomial(ell,2)*Phi_fork = $rhs")
println("match = $(lhs == rhs)")
```

```
Phi_identity + ell*Phi_inline = 9//10
1 - binomial(ell,2)*Phi_fork = 9//10
match = true
```

`mers_naive.jl`’s own $k==1/k==2$ regular-transmission branches charge exactly this $1 - \text{binomial}(\text{ell}_C, 2) / \text{binomial}(I_c, 2)$ term; upstream’s rewrite dropped it to zero.

Unlike bug 1, bug 2 is **state-dependent**: its size scales with the number of within-host births that occur while $\ell \geq 2$, so it biases parameter estimates (β_{cc} , β_{hh} , the initial conditions), not just an additive constant on the likelihood. Measured earlier

on a 4-camel-tip tree with long coexisting branches ($\beta_{cc}=3$, $N_c=40$, $I_{c0}=0.25$, 30 reps of $N_p=4000$):

kernel	logLik
pre-rewrite GuidedMERS (had both correct terms)	-28.63 ± 0.04
upstream rewrite, bug 1 fixed only	-22.58 ± 0.30
upstream rewrite, both fixes (this branch)	-28.58 ± 0.05

Fixing bug 1 alone moves the estimate the *wrong* way ($-28.6 \rightarrow -22.6$) because bug 2’s missing charge was still inflating it; both fixes together land back on the pre-rewrite reference value.

The full 274-tip tree: measured, not live

Aaron’s own `test/mers_guided.jl` filters the real 274-tip empirical MERS tree (Dudas et al. 2018) and fits it with 170 iterations of `mif`. That comparison takes about 5 minutes per kernel and needs a second, separate Julia project (upstream’s checkout, patched just to parse — see the appendix), so it is **not** re-executed live in this document. The numbers below were measured on 2026-09-19, same seed (2121916527), same guide (`fsmarkov(Camel=>0.5, Human=>0.5, (Camel,Human)=>0.01)`), same `pfilter/mif` calls in both cases:

stage	this branch (both fixes)	upstream (patched only to run)
initial pfilter, $N_p=1000$	–Inf	–19021.22
after <code>mif</code> , 120 iterations	–132.86	–213.68
after <code>mif</code> , 170 iterations (final)	–162.91	–409.36

The initial-pfilter row is one stochastic draw each and the two kernels consume different numbers of random draws per step (the fixes change how many `rcateg` calls a branch makes), so that row is not a controlled comparison — do not read “–Inf vs –19021” as evidence by itself. The load-bearing rows are the two post-`mif` ones: this branch reaches a substantially better final likelihood (–162.91 vs –409.36) and degrades far less from its own better checkpoint in the last stage (30 nats vs 196 nats). Fitted parameters end up in the same regime either way ($\beta_{cc} \approx 1.5$ – 1.7 , $\beta_{hh} \approx 2.5$ – 2.7 , $\chi_c = \chi_h = 1$), so the two runs are not diverging into different qualitative fits — the fixed kernel is fitting the same data better.

Appendix: reproducing the full-tree comparison

Not executed here (upstream’s checkout needs two out-of-band patches just to parse — a missing end closing module `GuidedMERSTest`, and a missing `Distributions` entry in `test/Project.toml`). From the repo root, against this branch:

```
using Test, Crayons
const h1 = crayon"bold blue"; const h2 = s -> crayon"!bold light_yellow"("- "s)
# RUN_HEAVY_TESTS=yes julia --project=test, from PhyloPOMP.jl/test/
include("mers_guided.jl") # runs pfilter then the full 170-iteration mif chain
```

Against upstream (../PhyloPOMP.jl_aarons, branch mers_guide, after Pkg.add("Distributions") to its test environment):

```
src = read("test/mers_guided.jl", String)
Base.include_string(Main, src * "\nend\n", "mers_guided.jl") # supplies the missing `e
```

Both print `logLik(pf)/logLik(mf)` if you add a `println` after each `@time` call — neither file has its own `@test` assertions on the value, so a bare “completed with no error” from either run means nothing on its own.

Summary

1. **Bug 1** (fork boost missing /2): exact, reproducible, confirmed independently by the compiler’s rational-arithmetic IR. A $\log 2$ penalty per within-host coalescence in the sampled coloring; a pure offset only when the data force every node’s host assignment, otherwise it tilts the fit toward cross-host transmission.
2. **Bug 2** (missing no-visible-fork factor on regular within-host births): confirmed by `mers_naive.jl`’s independent formula and the Chu-Vandermonde identity, checked live above. State-dependent — biases β_{cc} , β_{hh} , and the initial conditions, not just the likelihood level.
3. Fixing both, measured on the real 274-tip MERS tree with `mif`: final likelihood -162.91 vs upstream’s -409.36 , same seed, same guide, same number of iterations.

Both bugs are fixed only in this branch’s copy of `src/examples/mers_guided.jl`. Upstream (kingaa/PhyloPOMP.jl, commit 25039d0/44edaf6) still has both. See `handoffs/M14_upstream_guided_sync.md` for the full sync writeup and `handoffs/M14_review_protocol.md` for an independent-verification checklist covering these same claims.